Create Dataset

```
sales_data = [

(1001,"Laptop","Electronics","Hyderabad",2,65000,"Completed"),
(1002,"Mobile","Electronics","Bangalore",3,25000,"Completed"),
(1003,"Chair","Furniture","Mumbai",5,3500,"Pending"),
(1004,"Table","Furniture","Delhi",2,12000,"Completed"),
(1005,"Shoes","Fashion","Chennai",4,2500,"Completed"),
(1006,"Watch","Fashion","Pune",1,8000,"Cancelled"),
(1007,"TV","Electronics","Hyderabad",1,45000,"Completed"),
(1008,"Laptop","Electronics","Mumbai",2,65000,"Completed"),
(1009,"Chair","Furniture","Delhi",3,3500,"Pending"),
(1010,"Shoes","Fashion","Bangalore",5,2500,"Completed"),
(1011,"Mobile","Electronics","Chennai",4,25000,"Completed"),
(1012,"Watch","Fashion","Hyderabad",2,8000,"Completed"),
(1013,"TV","Electronics","Delhi",1,45000,"Completed"),
(1014,"Table","Furniture","Pune",3,12000,"Cancelled"),
(1015,"Laptop","Electronics","Bangalore",1,65000,"Completed"),
(1016,"Mobile","Electronics","Mumbai",2,25000,"Completed"),
(1017,"Shoes","Fashion","Delhi",4,2500,"Pending"),
(1018,"Chair","Furniture","Hyderabad",2,3500,"Completed"),
(1019,"TV","Electronics","Chennai",1,45000,"Completed"),
(1020,"Watch","Fashion","Bangalore",3,8000,"Completed")

]

columns = [
"order_id",
"product",
"category",
"city",
"quantity",
"price",
"status"
]

df = spark.createDataFrame(
    sales_data,
    columns
```

```
)

display(df)
```

Basic Exercises

1. Display all records.

2. Display schema.

3. Display first 10 records.

4. Select product and price columns.

5. Select city and category columns.

6. Count total records.

7. Count total columns.

8. Display distinct cities.

9. Display distinct categories.

10. Display distinct statuses.

Filter Exercises

11. Display Electronics products.

12. Display Furniture products.

13. Display Fashion products.

14. Display orders from Hyderabad.

15. Display orders from Bangalore.

16. Display Completed orders.

17. Display Pending orders.

18. Display Cancelled orders.

19. Display price greater than 30000.

20. Display quantity greater than 2.

21. Display Electronics orders from Hyderabad.

22. Display Furniture orders from Delhi.

23. Display Fashion orders from Bangalore.

24. Display orders from Hyderabad or Mumbai.

25. Display orders where price is between 10000 and 50000.

Sorting Exercises

26. Sort by price ascending.

27. Sort by price descending.

28. Sort by quantity descending.

29. Sort by city and price.

30. Sort by category and product.

Column Exercises

31. Create total_amount = quantity * price.

32. Create tax = total_amount * 0.05.

33. Create final_amount = total_amount + tax.

34. Create discount = total_amount * 0.10.

35. Create net_amount = total_amount - discount.

36. Rename price to unit_price.

37. Rename status to order_status.

38. Drop category column.

39. Drop city column.

40. Create a copy DataFrame with all calculated columns.

GroupBy Exercises

41. Count orders by city.

42. Count orders by category.

43. Count orders by status.

44. Find total revenue by city.

45. Find total revenue by category.

46. Find total revenue by product.

47. Find average product price by category.

48. Find maximum product price by category.

49. Find minimum product price by category.

50. Find total quantity sold by product.

Built-in Functions

51. Convert product names to uppercase.

52. Convert product names to lowercase.

53. Find length of product names.

54. Extract first 3 characters of product names.

55. Concatenate city and category.

56. Create city_category column.

57. Replace Electronics with Electronic Items.

58. Create product_code using substring.

59. Trim product names.

60. Display products containing 'a'.

when / otherwise

61. Create price_band:
    price >= 50000 -> Premium
    price >= 10000 -> Standard
    Else Budget

62. Count products by price_band.

63. Create order_size:
    quantity >= 4 -> Large
    Else Small

64. Count orders by order_size.

65. Create revenue_band:
    total_amount >= 100000 -> High
    Else Low

Spark SQL

66. Create temp view sales.
67. Display all records using SQL.
68. Count orders by city using SQL.
69. Count orders by category using SQL.
70. Find revenue by city using SQL.
71. Find revenue by category using SQL.
72. Find top 5 orders by revenue using SQL.
73. Find completed orders using SQL.
74. Find average price by category using SQL.
75. Find city with highest revenue using SQL.

Window Functions

76. Create row_number by revenue.
77. Create rank by revenue.
78. Create dense_rank by revenue.
79. Find top 5 revenue orders.
80. Find highest revenue order.
81. Find highest revenue order by city.
82. Find highest revenue order by category.
83. Create running total of revenue.

84. Use lag() on revenue.

85. Use lead() on revenue.

86. Compare current revenue with previous revenue.

87. Find revenue difference.

88. Find top 2 orders per city.

89. Find top 3 orders per category.

90. Generate revenue ranking report.

Mini ETL Exercises

91. Read source data.

92. Create calculated columns.

93. Filter completed orders.

94. Calculate revenue.

95. Generate city revenue report.

96. Generate category revenue report.

97. Save output as Parquet.

98. Read Parquet.

99. Create SQL view from Parquet.

100. Generate final business report.